

Marriage Hall Booking System

1. Introduction

The Marriage Hall Booking System is a simple C++ console-based project. It allows a user to view available marriage halls, view event packages, book a hall, save booking records in a file, view saved bookings, and add payment against a booking.

This project is designed to demonstrate the basic Object-Oriented Programming concepts required in the course:

- Classes
- Inheritance
- Polymorphism
- File Handling

The program uses only these header files:

```
#include <iostream>
#include <fstream>
#include <string>
```

iostream is used for input/output, **fstream** is used for file handling, and **string** is used to store text data.

2. Main Features

The project provides the following features:

- Show available marriage halls
- Show available packages
- Book a marriage hall
- Calculate total bill
- Enter advance payment
- Show paid amount and remaining balance
- Add more payment later using booking ID
- Save booking records in a file
- View saved bookings from file

3. OOP Concepts Used

3.1 Classes

The project is divided into different classes. Each class has its own responsibility.

Main classes used in the project:

- Person
- Customer
- EventPackage
- BasicPackage
- PremiumPackage
- RoyalPackage
- Hall
- Booking
- MarriageHallSystem
-

Example:

```
class Hall {  
    int id, capacity, rent;  
    string name;  
  
    public:  
    Hall(int hallId, string hallName, int hallCapacity, int hallRent) {  
        id = hallId;  
        name = hallName;  
        capacity = hallCapacity;  
        rent = hallRent;  
    }  
};
```

This class stores information about a marriage hall such as hall name, capacity, and rent.

3.2 Inheritance

Inheritance is used where one class gets properties and functions from another class.

In this project, Customer inherits from Person.

```

class Person {
protected:
    string name, phone;

public:
    void inputPerson() {
        cout << "Enter name: ";
        getline(cin, name);
        cout << "Enter phone: ";
        getline(cin, phone);
    }

    virtual void show() = 0;
};

class Customer : public Person {
    string cnic;

public:
    void inputCustomer() {
        inputPerson();
        cout << "Enter CNIC: ";
        getline(cin, cnic);
    }
};

```

Here, Customer can use the name and phone data from the Person class.

3.3 Polymorphism

Polymorphism is used through virtual functions. The base class EventPackage has virtual functions, and different package classes provide their own implementation.

```

class EventPackage {
public:
    virtual string getName() = 0;
    virtual int getPricePerGuest() = 0;
    virtual int getDecorationCharges() = 0;
};

```

Different package classes inherit this class:

```
class BasicPackage : public EventPackage {  
public:  
    string getName() { return "Basic Package"; }  
    int getPricePerGuest() { return 800; }  
    int getDecorationCharges() { return 20000; }  
};
```

The program uses a base class pointer to call the selected package at runtime:

```
EventPackage* package = createPackage(packageChoice);
```

This is an example of runtime polymorphism.

3.4 File Handling

File handling is used to store and read booking records.

The program uses ofstream to write booking data into a file:

```
ofstream file("marriage_bookings.txt");  
  
for (int i = 0; i < totalBookings; i++) {  
    bookings[i].writeToFile(file);  
}
```

The program uses ifstream to read saved booking records:

```
ifstream file("marriage_bookings.txt");  
string line;  
  
while (getline(file, line)) {  
    cout << line << endl;  
}
```

The file used in this project is:

marriage_bookings.txt

4. Class Explanation

4.1 Person Class

Person is an abstract base class. It stores common information such as name and phone number.

Important code:

```
class Person {  
protected:  
    string name, phone;  
  
public:  
    void inputPerson();  
    virtual void show() = 0;  
};
```

The function show() is pure virtual, so Person acts as an abstract class.

4.2 Customer Class

Customer inherits from Person. It adds CNIC information.

```
class Customer : public Person {  
    string cnic;  
  
public:  
    void inputCustomer();  
    string getName();  
    string getPhone();  
    string getCnic();  
    void show();  
};
```

This class is used when a new booking is created.

4.3 EventPackage Class

EventPackage is a base class for all packages.

```
class EventPackage {  
public:  
    virtual string getName() = 0;
```

```
        virtual int getPricePerGuest() = 0;
        virtual int getDecorationCharges() = 0;
    };
```

It defines common functions that every package must have.

4.4 Package Classes

The project has three package classes:

- BasicPackage
- PremiumPackage
- RoyalPackage

Example:

```
class RoyalPackage : public EventPackage {
public:
    string getName() { return "Royal Package"; }
    int getPricePerGuest() { return 1800; }
    int getDecorationCharges() { return 70000; }
};
```

Each package has a different price per guest and decoration charges.

4.5 Hall Class

The Hall class stores hall details.

```
class Hall {
    int id, capacity, rent;
    string name;
};
```

It stores:

- Hall ID
- Hall name
- Capacity
- Rent

4.6 Booking Class

The Booking class stores complete booking information.

```
class Booking {
    int bookingId, guests, totalBill, paidAmount;
```

```
Customer customer;  
Hall hall;  
string date, packageName;  
};
```

It stores:

- Booking ID
- Customer details
- Hall details
- Event date
- Number of guests
- Package name
- Total bill
- Paid amount
- Remaining balance

Payment is handled in this class:

```
int getBalance() {  
    return totalBill - paidAmount;  
}  
  
void addPayment(int amount) {  
    if (amount > getBalance()) {  
        cout << "Payment balance se zyada nahi ho sakti." << endl;  
    } else {  
        paidAmount = paidAmount + amount;  
    }  
}
```

4.7 MarriageHallSystem Class

This is the main controller class of the project. It manages halls, bookings, menu options, file saving, and payment updates.

```
class MarriageHallSystem {  
    Hall halls[3];  
    Booking bookings[50];  
    int nextBookingId, totalBookings;  
};
```

This class controls the full program flow.

5. Booking Process

The booking process works like this:

- User selects a hall.
- User enters customer details.
- User enters event date and number of guests.
- User selects a package.
- Program calculates total bill.
- User enters advance payment.
- Program shows booking details.
- Booking is saved in file.

Bill calculation:

```
int bill = selectedHall.getRent()  
        + package->getDecorationCharges()  
        + guests * package->getPricePerGuest();
```

This means:

Total Bill = Hall Rent + Decoration Charges + Guest Charges

6. Payment Feature

The payment feature allows the user to enter advance payment at the time of booking.

Example:

```
cout << "Enter advance payment: ";  
cin >> advancePayment;
```

The system also allows adding more payment later:

```
bookings[i].addPayment(amount);  
saveAllBookings();
```

After payment is added, the program updates:

- Paid amount
- Remaining balance
- Booking file record

7. File Handling Process

When a booking is created or payment is added, the system saves booking records in:

marriage_bookings.txt

The writing process is handled by:

```
void saveAllBookings() {  
    ofstream file("marriage_bookings.txt");  
  
    for (int i = 0; i < totalBookings; i++) {  
        bookings[i].writeToFile(file);  
    }  
}
```

The reading process is handled by:

```
void viewBookings() {  
    ifstream file("marriage_bookings.txt");  
    string line;  
  
    while (getline(file, line)) {  
        cout << line << endl;  
    }  
}
```

8. Menu Options

The program shows this menu:

```
===== Marriage Hall Booking System =====  
1. Show Halls  
2. Show Packages  
3. Book Hall  
4. View Bookings  
5. Add Payment  
0. Exit
```

Each option calls a different function in the MarriageHallSystem class.

Example:

```
if (choice == 1) showHalls();  
else if (choice == 2) showPackages();  
else if (choice == 3) bookHall();  
else if (choice == 4) viewBookings();  
else if (choice == 5) addPayment();
```

9. How to Compile and Run

Compile the program using:

```
g++ -std=c++14 MarriageHallBookingSystem.cpp -o MarriageHallBookingSystem.exe
```

Run the program using:

MarriageHallBookingSystem.exe

10. Conclusion

This project is a simple but complete C++ OOP-based Marriage Hall Booking System. It clearly demonstrates classes, inheritance, polymorphism, and file handling. The payment feature makes the system more practical because it can record advance payment, add further payment, and show the remaining balance.